

4. Security Protocols

The following key exchange protocol has been designed to allow A to guess and share a new (i.e. fresh) session key K with B . Server S is assumed to be trustworthy and the public key of S , namely K_S , is the only public key that A knows in advance.

1. $A \rightarrow S : A, B$
2. $A \leftarrow S : E(K_S^{-1}, [A, K_A]), E(K_S^{-1}, [B, K_B])$
3. $A \rightarrow B : E(K_B, [K, N_A])$
4. $A \leftarrow B : E(K, [K, N_A])$

where N_A is a nonce.

(a) Show that the protocol does not meet its design objectives by presenting an attack.

COME LEGGERE LE 4 PERMANENTI:

- 1 **Richiesta al Server:** $A \rightarrow S : A, B$
 - A contatta il Server fidato S per avviare la comunicazione, chiedendo essenzialmente i dati per sé (A) e per il destinatario (B).
- 2 **Rilascio dei Certificati:** $A \leftarrow S : E(K_S^{-1}, [A, K_A]), E(K_S^{-1}, [B, K_B])$
 - Il server S risponde ad A inviando le chiavi pubbliche di A (K_A) e B (K_B).
 - L'operazione $E(K_S^{-1}, \dots)$ significa che questi pacchetti sono firmati con la chiave privata del server. Fungono da veri e propri *certificati digitali*: A è certo che la chiave pubblica di B sia autentica e non manomessa.
- 3 **Invio della Chiave di Sessione:** $A \rightarrow B : E(K_B, [K, N_A])$
 - A genera una nuova chiave segreta di sessione (K) e un Nonce (N_A).
 - Li impacchetta e li cifra usando la chiave pubblica di B (K_B) ottenuta al passo 2. Così facendo, *solo ed esclusivamente* B (usando la sua chiave privata) potrà decifrare il pacchetto e scoprire K .
- 4 **Conferma di Ricezione:** $A \leftarrow B : E(K, [K, N_A])$
 - B decifra il messaggio 3, estrae la nuova chiave K e il Nonce N_A .
 - Per dimostrare ad A di avercela fatta, prende K e N_A e li cifra usando la nuova chiave di sessione K appena concordata. Inviandola ad A , B gli dimostra: "Ho aperto il tuo pacchetto e la nostra nuova chiave funziona".

PERCHÉ NON È SICURO: CI SONO DUE MOTIVI:

- 1) **REPLAY ATTACK:** NEL PASSAGGIO 3, A MANDA UN NONCE (N_A) MA QUESTO JENKE SOLO AD A . NON C'È NESSUN CONTROLLO (AD ESEMPIO NO) CHE DIMOSTRI A B CHE QUEL MESSAGGIO È RECENTE. UN HACKER POTREBBE INTERCETTARE IL MESSAGGIO 3, RITRASMETTERLO (REPLAY) A B IN UN SECONDO MOMENTO. B LO DECFIRA E LO ACCETTA.
- 2) **NESSUNA AUTENTICAZIONE:** NEL PASSAGGIO 3 IL MESSAGGIO È CIFRATO SOLO CON LA CHIAVE PUBBLICA DI B (K_B). MANDA LA FIRMA DIGITALE DI A . UN HACKER PUÒ INVENTARSI UNA CHIAVE IMPACCHETTANOLA CON UN FINTO NONCE, CIFRARE TUTTO CON LA CHIAVE PUBBLICA K_B E INVIARLO A B FACCENDO FINTA DI ESSERE A .

PER RENDERE SICURO MODIFICHIAMO IL PUNTO 3 COSÌ:

Ecco la scomposizione esatta della formula del prof pezzo per pezzo, perfetta per giustificare la risposta all'esame:

$$A \rightarrow B : \{C_A, \{K, N_A, T\}_{K_A^{-1}}\}_{K_B}$$

- T (Timestamp):** Inserisce l'orario. Risolve il Replay Attack garantendo la "freschezza" a B (il messaggio scade).
- $\{K, N_A, T\}_{K_A^{-1}}$ (Firma Digitale):** Cifra i dati con la chiave privata di A . Risolve l'impersonificazione garantendo a B l'autenticità del mittente.
- C_A (Certificato):** È il certificato digitale di A (ricevuto dal Server al passaggio 2). Il prof lo aggiunge furbamente per dare a B la chiave pubblica di A già validata dal server, permettendogli di verificare la firma subito.
- $\{\dots\}_{K_B}$ (Busta Esterna):** Cifra l'intero blocco con la chiave pubblica di B . Garantisce la confidenzialità totale: solo B può aprire il pacchetto esterno e leggere il contenuto.

K = CHIAVE DI SESSIONE

Consider the following protocol:

1. $C \rightarrow AS : C, S, N_C$
2. $AS \rightarrow C : AS, \{AS, C, NC, KS\}_{KAS}^{-1}$

where a client **C** wants to communicate with a server **S** and wants to be sure it is indeed doing it with **S**.

Since **C** does not have a public key of **S**, **C** asks an Authentication Server **AS** to get it.

Of course, we assume that **C** has the public key of **AS**, namely K_{AS} .

(a) Show that the protocol is insecure by describing how it can be attacked.

Il problema principale
è che nel messaggio 2
manca la firma del server S
quindi AS sta mandando una
chiave, ma non certifica di
che sia quella chiave.

Un attaccante intercetta il primo messaggio e modifica il secondo identificando col suo:

$$M \rightarrow AS : C, M, N_C$$

Il server di autenticazione risponde con la chiave pubblica dell'attaccante: $AS, \{AS, C, N_C, KM\}_{KAS}^{-1}$

Il client **C** stabilisce una connessione con **M** pensando di star parlando con **S**

Per risolvere: $\{AS, C, N_C, KS\}_{KAS}^{-1}$ data il client **C** ha anche l'identificativo del server **S** e l'attacco man-in-the-middle non è più possibile in quanto **C** conosce il nome associato alla chiave e trova che "E" al posto di "S" bloccerebbe subito la connessione.

Il controllo di coerenza (Come **C** smaschera **E**)

Devi immaginare il pacchetto firmato dall'Authentication Server (**AS**) come un documento di identità sigillato.

- L'aspettativa: Il client **C** ha iniziato il protocollo chiedendo esplicitamente: "Voglio la chiave per parlare con **S**". Quindi si aspetta che il documento di ritorno certifichi la chiave di **S**.
- La scoperta dell'inganno: Se l'attaccante **E** fa l'attacco Man-in-the-Middle e si fa generare la chiave, **AS** (che non sa dell'attacco) scriverà onestamente il nome di **E** nel certificato: $\{AS, C, E, N_C, KE\}_{KAS}^{-1}$.
- Il blocco: Quando **C** riceve e decifra il pacchetto, esegue un banale confronto logico: "Io ho chiesto la chiave per **S**, ma questo documento ufficiale mi dice che la chiave allegata qui dentro appartiene a **E**".

Conclusione

Visto che il nome richiesto non combacia con il nome certificato ($S \neq E$), **C** capisce istantaneamente che qualcuno ha manipolato la comunicazione. Rifiuta la chiave di **E** e fa cadere (blocca) la connessione prima di inviare qualsiasi dato segreto.

The **CRAM-MD5** (Challenge-Response Authentication Mechanism) protocol, used to provide authentication for the **IMAP4** email protocol, consists of the following steps:

1. $C \rightarrow S : C$ (Client sends a request to Server)
2. $S \rightarrow C : N || T || S$ (Server responds with Nonce, Timestamp, and Server identifier)
3. $C \rightarrow S : U, H(K || T)$ (Client sends Username and Hash of the Key and Timestamp)

Where:

- **C** = Client
- **S** = Server
- **N** = Nonce (a random number)
- **T** = Timestamp
- **U** = Username
- **K** = Cryptographic key derived from the user's password
- **H** = Cryptographic hash function (MD5)

The goal of the protocol is to allow **S** to authenticate **C**. It is assumed that **S** knows **K**.

1. Is the timestamp **T** necessary? Please justify your answer.
▶ Risposta
2. Is the nonce **N** necessary? Please justify your answer.
▶ Risposta

1) Sì il timestamp è necessario per evitare attacchi di replay e spoofing. Senza il timestamp un avversario potrebbe intercettare un precedente messaggio (intercepto), conservarlo valido per comunicazioni, in un secondo momento per ottenere un altro non autorizzato.

2) Non è strettamente necessario. Siccome non viene usato nella fase finale non ha alcun ruolo nel dimostrare al server la presenza del messaggio.

2023 Gennaio - Autenticazione e scambio di chiavi

3. Security Protocols

Consider the following authentication and key agreement protocol. Alice (A) and Bob (B) want to establish a session key using a long-term symmetric key K_{AB} . First Alice generates a nonce N_A and sends it along with her identity to Bob. Bob generates his own nonce N_B and sends it together with the encryption of Alice's nonce under the long-term key K_{AB} . Alice acknowledges receipt of this message by sending the encryption of Bob's nonce under the long-term key. Finally Bob generates the session key k and sends it to Alice encrypted under K_{AB} .

$A \rightarrow B: A, N_A$
 $A \leftarrow B: \{N_A\}_{K_{AB}}, N_B$
 $A \rightarrow B: \{N_B\}_{K_{AB}}$
 $A \leftarrow B: \{k\}_{K_{AB}}$

where both nonces and keys consists of 128-bit.

- (a) This protocol is flawed. Show how Eve could learn a session key that Alice thinks she has securely established with Bob.
- (b) Propose a way to fix the protocol to defend against this attack. Explain why your fix prevents this attack.

- ▶ Soluzione (a)
- ▶ Soluzione (b)

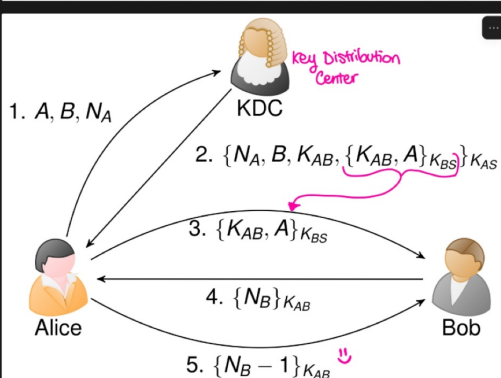
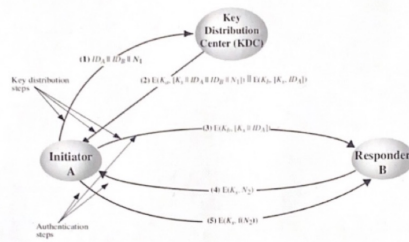
- a)
- Alice inizia il protocollo e manda in chiaro A, N_A .
 - Bob risponde con $\{N_A\}_{K_{AB}}, N_B$
 - Eve blocca il messaggio di Bob. Buina una N_B e lo sostituisce con una sua chiave k in chiaro. Invece ad Alice il pacchetto mangiato $\{N_A\}_{K_{AB}}, k$
 - Alice riceve quel pacchetto. Crede che quel k sia il normale nonce di Bob. Seguendo il passo 3 del protocollo, prende quel valore, lo cifra e lo restituisce $\{k\}_{K_{AB}}$
 - Eve cattura $\{k\}_{K_{AB}}$ e cioè rimanda indietro immediatamente finendo che sia il messaggio finale (passo 4) mandato da Bob
 - Alice decifra il messaggio, vede che tutto torna e accetta k come chiave di sessione sicura.

b) Si modifica il passo 2 così: $B \rightarrow A: \{N_A, N_B\}_{K_{AB}}$

2022 Luglio - Protocollo NSSK

3. Security Protocols

What is the main purpose of the last two messages in the Needham-Schroeder symmetric key protocol?



Secondo ai due utenti Alice e Bob che partecipano alla comune sessione di autenticazione a vicenda

Consider the following authentication protocol

$$1. A \rightarrow B : E(K, [A, B, T])$$

where K is a key shared between A and B and T is a time-stamp generated by A .

- a. Indicate which of the following security properties are met by the protocol. For the properties that are met by the protocol describe the assumptions that must hold for the property to hold. For the properties that are not met by the protocol write an execution trace witnessing the violation of the property.
- A authenticates B
 - B authenticates A
 - Secrecy of N
 - Secrecy of K

a) A NON PUO' AUTENTICARE B IN QUANTO NON RICEVE NESSUN TIPO DI NOTIFICA DA B

b) SI DEVE ASSUMERE CHE K SIA CONDIVISA E CONOSCIUTA SOLO DA A E B , INALTRA IL TIME STAMP DEVEREBBE SUFFICIENTEMENTE RECENTE PER EVITARE REPLAY ATTACKS

c) NON C'E' NESSUNA N IMPLICATA NEL MESSAGGIO

d) K DEVE ESSERE CONOSCIUTA SOLO DA A E B E NON FACILMENTE DETERMINABILE CONOSCENDO IL PROTOCOLLO ED IL CONTENUTO DEI MESSAGGI SCAMBIATI.

a. Describe the actions carried out by B upon receipt of the message for a proper functioning of the protocol.

► Risposta

c. What are the consequences of using an implementation of the protocol where the timestamp is very coarse, e.g. successive time instants correspond to large intervals (say several seconds).

► Risposta

d. Assume that successive time instants correspond to 1 second and that timestamps are represented by 16 bits. How long must an attacker wait to mount a replay attack?

b) DEJENNI LE AZIONI COMPATTE DA B ALLA RICEZIONE DEL MESSAGGIO PER IL CORRETTO FUNZIONAMENTO DEL PROTOCOLLO.

B USA LA CHIAVE CONDIVISA K PER DECRYPTARE IL MESSAGGIO E LEGGERE IL CONTENUTO, VERIFICHERA' CHE IL PRIMO IDENTIFICATIVO INDICHERA' IL MANDANTE (A) IL SECONDO IDENTIFICATIVO INDICHERA' CHE IL MESSAGGIO ERA DESTINATO A LUI, Ossia B , E CHE IL TIME STAMP T SIA SUFFICIENTEMENTE RECENTE PER ASSICURARE LA FIDABILITA' DEL MESSAGGIO.

c) SE UN ANTAGONISTA INTERCETTA IL MESSAGGIO POTREBBE RITRARRARLO PER REPLAY ATTACK.

SE FOSSERO STATI 4 SECONDI AVREI FATTO
 $2^{16} \cdot 4$

d) L'ANTAGONISTA DOVRA' ASPETTARE 2^{16} SECONDI ASSUNTO CHE IL TIME STAMP NON HA AD AVERE LO STESSO VALORE ED ESISTE ALCUNO VALORE PER DESTINATARIO.

The following protocol has been designed to generate and distribute a session key K between

A and B using S as key distribution center:

- $A \rightarrow S : E(K_{AS}, B)$
- $S \rightarrow A : E(K_{AS}, [K, E(K_{BS}, [A, K])])$
- $A \rightarrow B : E(K_{BS}, [A, K])$

where K_{AS} is a symmetric key known only to A and S and K_{BS} is a symmetric key known only to B and S .

The protocol is vulnerable to a replay attack.

a. Describe an attack.

► Soluzione

b. Assume that A , B and S have synchronized clocks. Modify the protocol so to prevent replay attacks by adding values to the messages of the protocol. Please justify your answers.

► Soluzione

a) IL PROBLEMA PRINCIPALE E' LA MANCANZA DI VALORE O TIME STAMP CHE GARANTISCONO LA FIDABILITA' DEI DATI. UN ANTAGONISTA POTREBBE ORIGINARE IL TERZO MESSAGGIO DATO CHE NON CONTERREBBE INFORMAZIONI RIGUARDANDO LA SUA PRESENZA, PUO' ESSERE RITRARRATO IN UN SECONDO MOMENTO PER RIMANDARE A B UNA CHIAVE CONDIVISA K VERIFICA COSI' DA CONVINCRELA A CREARE FIDUCIA CON B , AUMENTANDO LA POSSIBILITA' DI FRODARE K CON UN ALTRO DI CINDO ANZICHIA.

Il vantaggio strategico per l'hacker (Attacco contro B):

L'obiettivo dell'hacker è costringere B a usare una chiave di sessione K vecchia e debole (che l'hacker ha già avuto il tempo di craccare).

1. L'hacker registra un Messaggio 3 legittimo avvenuto mesi fa.
2. Oggi, l'hacker si finge A e invia quel vecchio Messaggio 3 a B.
3. B lo riceve, usa la sua chiave K_{BS} e lo apre con successo. Poiché dentro legge il nome di "A" e non c'è nessun timestamp a dirgli che il messaggio è vecchio di mesi, B pensa: "Perfetto, A vuole parlare con me ora usando questa chiave K".

b)

1. Cerca la "Merce di Valore" (La Chiave di Sessione)
Gli attacchi mirano sempre al cuore del protocollo, ovvero la chiave di sessione (K). La primissima cosa da fare è guardare in quali messaggi viene trasportata questa chiave.

- o In questo protocollo, K viaggia nel Messaggio 2 (il Server la consegna ad A) e nel Messaggio 3 (A la consegna a B).
- o Quelli sono i tuoi due bersagli da analizzare.

2. Cerca la "Data di Scadenza" (Test di Freschezza)
Ora prendi i bersagli che hai individuato e guardaci dentro. I messaggi crittografici, per essere sicuri contro i Replay Attack, devono avere una "data di scadenza": un Nonce (N) o un Timestamp (T).

- o Guarda il Messaggio 2: $E(K_{AS}, [A, K])$. C'è un Nonce? No. C'è un Timestamp? No.
- o Conclusione immediata: Essendo privo di indicatori di tempo, se lo registro oggi e lo reinvio a Bob tra un anno, Bob lo aprirà e accetterà la vecchia chiave. Vulnerabilità trovata!
- o Guarda il Messaggio 3: $E(K_{BS}, [K, E(...)])$. Anche qui, niente N e niente T .
- o Conclusione: Se Alice fa una nuova richiesta, l'attaccante può bloccare il Server e rispondere inviandole questo vecchio messaggio 2. Alice lo accetterà senza sospettare nulla. Seconda vulnerabilità trovata!

3. Cerca il "Destinatario" (Test di Binding)
Un'altra cosa da controllare sempre (anche se in questo esercizio specifico il problema era la freschezza) è a chi è destinata la chiave.

- o Se A chiede al server una chiave per comunicare con B, quando il server risponde mandando la chiave, deve includere il nome di B nel pacchetto cifrato. Se non c'è il nome, un attaccante potrebbe farti credere che quella chiave serva per parlare con B, quando in realtà è per parlare con lui (Man-in-the-Middle).

$$S \rightarrow A: E(K_{AS}, [K, E(T(K_{BS}, [A, T, K]))])$$

$$A \rightarrow B: E(K_{BS}, [A, K, T])$$

2009 Luglio - Autenticazione

Si consideri il seguente protocollo per l'autenticazione di due agenti in possesso di una chiave simmetrica condivisa K .

1. $A \rightarrow B: N \oplus K$
2. $B \rightarrow A: N$

Il primo agente A genera una nonce N della stessa lunghezza di K , la mette in $\oplus$ con K e la invia al secondo agente B .

Quando B riceve il messaggio da A , lo mette in $\oplus$ con la chiave K e manda la stringa di bit risultante indietro ad A .

Se la stringa di bit ricevuta da A coincide con la nonce generata inizialmente, allora A può concludere di aver autenticato B .

- a. Si indichino almeno due proprietà che sono fondamentali per la sicurezza di un protocollo di autenticazione di questo tipo.
 - Soluzione
- b. Si discuta se le proprietà indicate sono soddisfatte dal protocollo in questione.
 - Soluzione

- a) • Autenticazione di B nel contesto di A
• Sicurezza della chiave K

- b) La chiave K viene compromessa. L'hacker non può (N) e i messaggi scambiati durante l'esecuzione del protocollo si ottiene K. $(N \oplus K) \oplus N = K$

ATTACCO MAN-IN-THE-MIDDLE

$$A \rightarrow M: \{A, N_A\}_{K_A}$$

M decide il messaggio, genera N_A e lo inoltra a Bob scambiandosi per Alice.
 $M \rightarrow B: \{A, N_A\}_{K_B}$

B $\rightarrow A: \{N_A, N_B\}_{K_A}$ il mantenimento M lo intercetta ma non può leggerlo. Tutto ciò che viene cifrato con una chiave pubblica (la chiave pubblica solo ed esclusivamente con la corrispondente chiave privata. Quindi M al momento non può essere né NA né NB. M quindi lo inoltra alla vera Alice. Alice lo decifra con la sua chiave privata, vede NA e crede cecatinamente che sia la risposta di M al

Si consideri il protocollo di autenticazione a chiave pubblica di Needham Schroeder:

1. $A \rightarrow B: \{A, N_A\}_{K_B}$
2. $B \rightarrow A: \{N_A, N_B\}_{K_A}$
3. $A \rightarrow B: \{N_B\}_{K_B}$

Come visto a lezione tale protocollo è vulnerabile ad un attacco man-in-the-middle.

Dire quali delle seguenti varianti del protocollo impediscono tale attacco e quali invece continuano a soffrirne. Si giustificano le risposte date.

► Spoiler attacco MITM

1.
 - a. $A \rightarrow B: \{A, B, N_A\}_{K_B}$
 - b. $B \rightarrow A: \{N_A, N_B\}_{K_A}$
 - c. $A \rightarrow B: \{N_B\}_{K_B}$

► Soluzione
2.
 - a. $A \rightarrow B: \{A, B, N_A\}_{K_B}$
 - b. $B \rightarrow A: \{N_A, N_B, B\}_{K_A}$
 - c. $A \rightarrow B: \{N_B\}_{K_B}$

► Soluzione
3.
 - a. $A \rightarrow B: \{A, B, N_A\}_{K_B}$
 - b. $B \rightarrow A: \{N_A, N_B\}_{K_A}$
 - c. $A \rightarrow B: \{A, B, N_B\}_{K_B}$

► Soluzione

Il primo messaggio. Come da protocollo, conferma la ricezione inviando il nonce A M: $A \rightarrow M: \{N_A\}_{K_A}$

Qua M decide il pacchetto e genera N_B in chiaro, lo cifra con la chiave di Bob e questo invia: $M \rightarrow B: \{N_B\}_{K_B}$

Per non essere rastato, manda il messaggio 2 a Bob e lo decifra di Bob: $\{B, N_A, N_B\}_{K_A}$.

1.
 - a. $A \rightarrow B : \{A, B, N_A\}_{K_B}$
 - b. $B \rightarrow A : \{N_A, N_B\}_{K_A}$
 - c. $A \rightarrow B : \{N_B\}_{K_B}$

▼ Soluzione

L'attacco è ancora possibile, all'attaccante C basta levare l'identificatore B al primo messaggio dopo averlo spaccettato con la sua chiave K_C .
2.
 - a. $A \rightarrow B : \{A, B, N_A\}_{K_B}$
 - b. $B \rightarrow A : \{N_A, N_B, B\}_{K_A}$
 - c. $A \rightarrow B : \{N_B\}_{K_B}$

▼ Soluzione

L'attacco non è più possibile in quanto nel messaggio 2 A si renderà conto che l'identificativo di B sarà diverso da quello di C con cui credeva di star comunicando.
3.
 - a. $A \rightarrow B : \{A, B, N_A\}_{K_B}$
 - b. $B \rightarrow A : \{N_A, N_B\}_{K_A}$
 - c. $A \rightarrow B : \{A, B, N_B\}_{K_B}$

▼ Soluzione

L'attacco è ancora possibile per le stesse ragioni del punto 1, l'attaccante, spaccettando i messaggi provenienti da A, li può liberamente manipolare e rimuovere gli identificativi di B.

In realtà basta mettere B dentro al messaggio 2, nel 1 non è necessario.

2008 Giugno - Autenticazione e Diffie-Hellman

1. Si consideri il seguente protocollo di sicurezza P per l'autenticazione tra due agenti A e B :

```

1.  $A \rightarrow B : N_a$ 
2.  $B \rightarrow A : \{N_a\}_{K_{ab}}, N_b$ 
3.  $A \rightarrow B : \{N_b\}_{K_{ab}}$ 
  
```

dove N_a e N_b sono nonces e K_{ab} è una chiave condivisa tra A e B .

- a. Si mostri che il protocollo non garantisce la mutua autenticazione.

► Soluzione

- b. Una delle regole di prudent engineering per la progettazione di protocolli di sicurezza presentate a lezione suggerisce di includere nei messaggi l'identità del mittente e del destinatario. Si mostri come l'applicazione di questo principio può prevenire l'attacco trovato al punto precedente.

► Soluzione

- c. Un ulteriore metodo per cercare di rendere sicuro il protocollo e quello di modificarlo nel seguente modo:

```

1.  $A \rightarrow B : N_a$ 
2.  $B \rightarrow A : \{N_a - 1\}_{K_{ab}}, N_b$ 
3.  $A \rightarrow B : \{N_b - 1\}_{K_{ab}}$ 
  
```

Si discuta se questo protocollo garantisce o meno la proprietà di mutua autenticazione.

► Soluzione

2. Si dimostri che il protocollo per scambio di chiavi di Diffie-Hellman non garantisce l'autenticazione.

► Soluzione

LA MUTUA AUTENTICAZIONE è una proprietà di sicurezza che si verifica quando entrambi le parti comunicano in una comunicazione reciproca. A verificarsi è contemporaneamente l'identità desiderata.

3) IL PROTOCOLLO NON GARANTISCE LA PROPRIETÀ DI AUTENTICAZIONE IN QUANTO VUOLIAMO AL SEGRETO MEMORIAMO ANCHE:

(S1) 1. $a \rightarrow i : m_a$ L'utente a lascia la stanza, vuole parlare con b e cerca un nonce m_a , viene intercettato dall'attaccante i.

(S2) 1. $a \leftarrow i : m_a$ L'attaccante i cancella un attimo la sessione 1. Apre una nuova sessione verso a e ci manda esattamente lo stesso m_a .

(S2) 2. $a \rightarrow i : \{m_a\}_{k_{ab}}, m_b$ Qui di cose in transito, a riceve m_a dal punto precedente e pensa che b vuole iniziare una nuova conversazione burrasca di giorno $\{m_a\}_{k_{ab}}, m_b$.

(S1) 2. $a \leftarrow i : \{m_b\}_{k_{ab}}, m_b$ L'attaccante riprende la sessione 1, eliminando prende il pacchetto creato appena quando inizia sessione 2 e lo invia come risposta nella sessione 1.

(S1) 3. $a \rightarrow i : \{m_b\}_{k_{ab}}$ a riceve la risposta nella sessione 1, vede che il suo m_b originale è stato cifrato correttamente e pensa ci sia il vero b.

- b)
- $A \rightarrow B: A, B, N_2$
 - $B \rightarrow A: \{A, B, N_2\}_{K_{AB}}, N_6$
 - $A \rightarrow B: \{A, B, N_6\}_{K_{AB}}$

c) Abbiamo lo stesso problema di prima. Solo che ora volta abbiamo $N_2, -1$

d) Il protocollo Diffie-Hellman permette a due persone di creare una chiave segreta *max-aria*, ma non verifica l'identità di chi sta partecipando. Questo lo rende ad un attacco *Man-in-the-middle*.

Ecco la dimostrazione spiegata in poche parole (perfetta per l'esame):

- 👁️ **L'Intercettazione:** A invia il suo valore matematico a B. Un intruso (*I*) intercetta la comunicazione e la blocca.
- 🗨️ **La Sostituzione:** *I* invia a B il *proprio* valore matematico, fingendo di essere A.
- 📩 **Il Ritorno:** B riceve il parametro (credendo sia di A), calcola la chiave e invia la sua risposta. *I* intercetta anche questa, la sostituisce con la propria e la inoltra ad A.
- 🌟 **Il Risultato Finale:** Si creano due chiavi segrete separate: una tra A e *I*, e una tra B e *I*.

👤 **Conclusione:**

A e B credono di aver concordato una chiave sicura tra loro, ma in realtà l'hacker è nel mezzo e può decifrare, leggere e ricifrare tutto il traffico. Poiché in Diffie-Hellman non ci sono password, firme o certificati, il protocollo **non offre alcuna mutua autenticazione**.

Alm dopo per rispondere alla d):

- 🟢 **Avvio Sessione 1:** L'attaccante *I* (fingendosi *A*) contatta *B* inviando un nonce a caso N_1 .
 $I(A) \rightarrow B: N_1$
 - 🟡 **Sfida di Bob:** *B* risponde come da protocollo.
 $B \rightarrow I(A): \{N_1\}_{K_{AB}}, N_6$
(Ora *I* deve assolutamente trovare il modo di calcolare $\{N_6\}_{K_{AB}}$ per finire l'inganno).
 - 🔴 **Avvio Sessione 2 (il trucco):** *I* lascia in pausa la prima connessione e ne apre una nuova con *B*, sempre fingendosi *A*, ma questa volta invia proprio il nonce N_1 appena ricevuto da Bob.
 $I(A) \rightarrow B: N_1$
 - 🟣 **L'oracolo Bob:** *B* elabora la Sessione 2 in modo innocente. Cifra il nonce ricevuto (che è il suo stesso N_1 della prima sessione) e risponde.
 $B \rightarrow I(A): \{N_1\}_{K_{AB}}, N_6'$
 - ⚡ **Scacco Matto:** *I* afferra $\{N_6\}_{K_{AB}}$ sfornato da Bob nella Sessione 2, lo inserisce come terzo messaggio della Sessione 1 e glielo invia!
 $I(A) \rightarrow B: \{N_6\}_{K_{AB}}$
- 👤 **Risultato:** Bob verifica il pacchetto, vede che è corretto e conclude con successo la mutua autenticazione convinto di parlare con Alice. In realtà ha parlato con l'attaccante e ha calcolato da solo la sua stessa password!

2007 Giugno - Scambio di chiavi

5. Protocolli di Sicurezza

Si consideri il seguente protocollo P_1 per la creazione di una chiave condivisa tra due agenti A e B:

- $A \rightarrow B: \{A, Na\}_{K_b}$
- $B \rightarrow A: \{B, Nb\}_{K_a}$
- $A \rightarrow B: \{zero, Msg\}_{Na \oplus Nb}$
- $B \rightarrow A: \{one, Msg\}_{Na \oplus Nb}$

dove *zero* e *one* sono identificatori distinti e $Na \oplus Nb$ è lo XOR bit a bit di Na e Nb .

- a. Si descrivano i singoli passi del protocollo e le proprietà di sicurezza per il quale è stato presumibilmente progettato.

► Risposta

- b. Si discuta se il protocollo garantisce o meno le proprietà di sicurezza indicate nella risposta alla domanda (a). Si supponga che la crittografia sia perfetta (ovvero non è soggetta ad attacchi di crittoanalisi).

► Risposta

a) • **Mutua Autenticazione:** I passi 1 e 2 servono ad Alice e Bob per scambiarsi due nonce segreti ($N_2 \in N_6$) usando 1 e 2 rispettive chiavi pubbliche, solo loro potranno leggerle.

• **Confidenzialità:** Nei passi 3 e 4, mandano i due nonce segreti e li fondono insieme con uno *xor* ($N_2 \oplus N_6$) per creare una nuova chiave per scambiarsi l'informazione. Usando questa chiave per proteggere il messaggio *msg*.

b) La presenza degli identificatori *zero* e *one* nel messaggio inviati ai passi 3 e 4 rende difficile la realizzazione di un attacco *man-in-the-middle*.

🔴 Lo scenario vulnerabile (Senza `zero` e `one`)

Immagina che i passi 3 e 4 siano identici:

3. $A \rightarrow B : \{Msg\}^{Na \oplus Nb}$

4. $B \rightarrow A : \{Msg\}^{Na \oplus Nb}$

Un attaccante (chiamiamolo Mallory) si mette in mezzo alla rete e fa un banale **Reflection Attack**:

- o Alice genera il messaggio 3 e lo invia.
- o Mallory lo intercetta e non lo fa arrivare a Bob. Mallory non ha la chiave e non può leggere *Msg*, ma non gli interessa. Prende l'intero "pacchetto chiuso" e lo rimbalza indietro ad Alice, spacciandolo per il messaggio 4 in arrivo da Bob.
- o Alice riceve il pacchetto, lo decifra con la sua chiave di sessione, ottiene *Msg* e pensa: *"Perfetto, Bob mi ha risposto e la comunicazione funziona!"*.
- o **Disastro**: Mallory ha completato l'autenticazione impersonando Bob, senza mai aver dovuto violare la crittografia.

🛡️ Lo scudo (Con `zero` e `one`)

Inserendo queste due semplici paroline, stiamo "etichettando" i messaggi per dare loro una **direzione specifica**.

0. $A \rightarrow B : \{zero, Msg\}^{Na \oplus Nb}$

1. $B \rightarrow A : \{one, Msg\}^{Na \oplus Nb}$

Vediamo come fallisce miseramente l'attacco di Mallory ora:

- o Alice invia il messaggio 3 verso Bob.
- o Mallory lo intercetta e, come prima, lo riflette indietro ad Alice sperando di ingannarla.
- o Alice lo decifra. Attenzione però: siccome Alice sta aspettando il messaggio 4 da Bob, il suo software è programmato per esigere che la prima parola dentro al pacchetto sia `one` .
- o Alice decifra il pacchetto rimbalzato e legge: `zero` .
- o **Vittoria**: Alice capisce istantaneamente che quello non è un messaggio di Bob, ma è il suo stesso pacchetto che le è tornato indietro come un boomerang. Rileva l'attacco e fa cadere la connessione.

🔒 2. Le parole `zero` e `one` nei messaggi 3 e 4 (Contro i Reflection Attack):

I passi 3 e 4 usano la **stessa chiave simmetrica** ($Na \oplus Nb$).

- o **L'attacco**: Senza le parole `zero` e `one` , un attaccante potrebbe intercettare il messaggio 3 generato da Alice ($\{Msg\}^{Na \oplus Nb}$) e "rifletterlo" (rimbalzarlo) indietro ad Alice spacciandolo per il messaggio 4. Alice lo accetterebbe!
- o **La toppa**: Inserendo `zero` nel msg 3 e `one` nel msg 4, i pacchetti diventano strutturalmente diversi. Se l'attaccante riflette il messaggio 3 ad Alice, lei lo decifra, legge la parola `zero` , capisce che è il suo stesso messaggio rimbalzato (perché si aspettava `one`) e blocca l'attacco.

2006 Luglio - Protocollo

6. Protocolli di Sicurezza

Si consideri il seguente protocollo P_1 utilizzato in ambito di telefonia mobile

- (1) $B \rightarrow M : B, K_B$
- (2) $M \rightarrow B : \{K\}_{K_B}$
- (3) $M \rightarrow B : \{M, P\}_K$

dove M è un terminale mobile con limitate capacità di calcolo, B è la stazione base, K_B è una chiave pubblica di B non nota a M , K è una chiave segreta per crittografia simmetrica generata da M e P una password nota solo a M e a B . Ogni qualvolta viene completato il protocollo la stazione B addebita a M un costo prestabilito.

- a. Si descrivano i singoli passi del protocollo e le proprietà di sicurezza per il quale è stato presumibilmente progettato.

 Risposta

2) - PASSO 1: $B \rightarrow M : B, K_B$

LA STAZIONE BASE B INVIÀ IN CHIARO IL PROPRIO IDENTIFICATIVO E LA SUA PROPRIA CHIAVE PUBBLICA K_B AL TERMINALE MOBILE M .

- PASSO 2: $M \rightarrow B : \{K\}_{K_B}$

M GENERA UNA CHIAVE DI SESSIONE SIMMETRICA K , LA CIFRA USANDO LA CHIAVE PUBBLICA K_B APPENA RICEVUTA E LA INVIÀ A B .

- PASSO 3: $M \rightarrow B : \{M, P\}_K$

M INVIÀ IL PROPRIO IDENTIFICATIVO E LA PASSWORD P CIFRANDO IL TUTTO IN MODA APPROPRIATA CON LA CHIAVE SIMMETRICA DI SESSIONE K

LE PROPRIETÀ CHE DOVREBBE VERIFICARSI SONO: - **CONFIDENZIALITÀ**: L'IDEA SEMPLICE DI CREDERE SEGRETA K

- **AUTENTICAZIONE** DI M .

- b. Spiegare una ragione per cui il protocollo P_1 è preferibile al seguente protocollo P_2 che, pur garantendo le stesse proprietà di sicurezza, è indiscutibilmente più semplice.

- (1) $B \rightarrow M : B, K_B$
- (2) $M \rightarrow B : \{K, M, P\}_{K_B}$

 Risposta

IL PROTOCOLLO P_1 È PREFERIBILE PERCHÉ ALCUNA IL VIOLAZIONE DELLE "LIMITATE CAPACITÀ" DI CALCOLO DEL TERMINALE M .

- c. Si discuta se il protocollo garantisce o meno le proprietà di sicurezza indicate nella risposta alla domanda (a).

IL PROTOCOLLO NON GARANTISCE LE PROPRIETÀ DI SICUREZZA IN QUANTO È POSSIBILE L'ATTACCO MAN-IN-THE-MIDDLE.

POICCHÉ K_B VIAGGIA IN CHIARO AL PASSO 1 E NON È LEGATA A UN CERTIFICATO DIGITALE, M ACCENDO QUALSIASI CHIAVE. UN AGNACCIANTE È PIÙ INGENUO IL MESSAGGIO È INVIATO A M LA PROPRIA CHIAVE PUBBLICA K_B . AL PASSO 2 M CREA LA CHIAVE DELL'AGNACCIANTE (K_E) PER CIFRARE LA CHIAVE DI SESSIONE K . L'AGNACCIANTE POTRÀ COSÌ DECCIFRARE K E USARE IN CHIARO LA PASSWORD P INVIATA AL PASSO 3.

- d. Nel caso sia stata individuata una vulnerabilità, si identifichi un raffinamento del protocollo che non soffra di tale vulnerabilità.

A tal fine si assuma che:

- B sia in possesso di un certificato $\{B, K_B\}_{K_{CA}^{-1}}$ emesso da un'autorità di certificazione CA relativo alla propria chiave pubblica K_B e
- M sia in possesso di K_{CA} , ovvero della chiave pubblica di CA .

1) $B \rightarrow M : N_B, \{B, K_B\}_{K_{CA}^{-1}}$

2) $M \rightarrow B : \{K\}_{K_B}$

3) $M \rightarrow B : \{N_B, M, P\}_K$

2006 Giugno - Autenticazione

5. Protocolli di Sicurezza

Si consideri il seguente protocollo:

- (1) $B \rightarrow A : B$
- (2) $A \rightarrow B : \{N_a\}_{K_{ab}}$
- (3) $B \rightarrow A : \{f(N_a)\}_{K_{ab}}$

dove K_{ab} è una chiave segreta condivisa da A e B , N_a è un numero generato con un generatore di numeri pseudo-casuali, e f è una funzione nota sia ad A che a B .

Si assuma che l'agente che esegue il protocollo impersonando il ruolo di A non possa eseguire lo stesso protocollo impersonando il ruolo di B . Ad esempio, B potrebbe essere il ruolo giocato dal telecomando della vostra macchina, mentre A quello giocato dal sistema di apertura installato sulla vostra macchina.

Si discuta perché la sicurezza del protocollo dipende dal numero di bit utilizzati per rappresentare il numero N_a .

● **Concetto chiave:** La sicurezza dipende dallo spazio delle combinazioni possibili per il nonce (2^n , dove n è il numero di bit).

🗺️ **Il problema dei pochi bit (Collisioni):** Se N_a è corto, il sistema esaurirà in fretta i numeri disponibili. Questo porterà inevitabilmente A a generare un valore N_a già utilizzato in una sessione passata.

👤 **La vulnerabilità (Replay Attack):** Un attaccante in ascolto passivo può intercettare e registrare un certo numero di coppie legittime di sfida e risposta, ovvero i pacchetti $\{N_a\}_{K_{ab}}$ e $\{f(N_a)\}_{K_{ab}}$.

✳️ **L'attacco:** Non appena la macchina A invia un N_a che l'attaccante ha già visto nel suo storico, egli può impersonare il telecomando B . Gli basterà semplicemente inviare in risposta il pacchetto $\{f(N_a)\}_{K_{ab}}$ memorizzato in precedenza. L'attaccante riesce così ad autenticarsi e ad aprire la macchina senza possedere la chiave segreta K_{ab} .

✅ **Conclusione:** Utilizzando un numero elevato di bit, la probabilità di ripetizione del nonce N_a diventa matematicamente trascurabile, neutralizzando del tutto l'utilità delle intercettazioni passate dell'attaccante.

ENTRA: PER MIGLIORARE IL SISTEMA POSSIAMO AGGIUNGERE UN TIME-STAMP. DIVISI:

- 1 $B \rightarrow A : B$
- 2 $A \rightarrow B : \{N_a\}_{K_{AB}}$
- 3 $B \rightarrow A : \{T_b, T_b\}_{K_{AB}}$

2006 Giugno - Autenticazione

5. Protocolli di Sicurezza

Si consideri il seguente protocollo:

- (1) $B \rightarrow A : B$
- (2) $A \rightarrow B : \{N_a\}_{K_{ab}}$
- (3) $B \rightarrow A : \{f(N_a)\}_{K_{ab}}$

dove K_{ab} è una chiave segreta condivisa da A e B , N_a è un numero generato con un generatore di numeri pseudo-casuali, e f è una funzione nota sia ad A che a B .

Si assuma che l'agente che esegue il protocollo impersonando il ruolo di A non possa eseguire lo stesso protocollo impersonando il ruolo di B . Ad esempio, B potrebbe essere il ruolo giocato dal telecomando della vostra macchina, mentre A quello giocato dal sistema di apertura installato sulla vostra macchina.

Si discuta perché la sicurezza del protocollo dipende dal numero di bit utilizzati per rappresentare il numero N_a .

► Risposta

6. Protocolli di Sicurezza

Nel seguente protocollo K_A^1 e K_B^1 sono chiavi pubbliche già note, mentre K_A^2 e K_B^2 sono nuove chiavi pubbliche. N_A e N_B sono nonces.

- M1. $A \rightarrow B : \{K_A^2, A\}_{K_B^1}$
- M2. $B \rightarrow A : \{N_B, K_B^2\}_{K_A^1}$
- M3. $A \rightarrow B : \{N_B, N_A\}_{K_B^2}$
- M4. $B \rightarrow A : \{N_A\}_{K_A^2}$

- a. Si descrivano i singoli passi del protocollo e le proprietà di sicurezza per il quale è stato presumibilmente progettato.
 - Risposta
- b. Si discuta se il protocollo garantisce o meno le proprietà di sicurezza descritte alla domanda precedente.
 - Risposta
- c. Nel caso sia stata individuata una vulnerabilità, si identifichi un raffinamento del protocollo che non soffra di tale vulnerabilità.
 - Soluzione

a. Descrizione dei passi e proprietà di sicurezza

Questo protocollo ha l'obiettivo di stabilire una mutua autenticazione tra due entità (A e B) e di scambiare in modo sicuro nuove chiavi pubbliche (di sessione o effimere, K_A^2 e K_B^2) e dei nonces (N_A , N_B).

✳️ Singoli passi del protocollo:

1. $M1. A \rightarrow B : \{K_A^2, A\}_{K_B^1}$
 A inizia la comunicazione inviando a B la sua nuova chiave pubblica K_A^2 e il suo identificatore A , cifrando tutto con la chiave pubblica "nota" di B (K_B^1). Solo B può leggere questo messaggio.
2. $M2. B \rightarrow A : \{N_B, K_B^2\}_{K_A^1}$
 B risponde generando un nonce N_B e la sua nuova chiave pubblica K_B^2 . Cifra questi dati con la chiave pubblica "nota" di A (K_A^1). Solo A può leggerli.
3. $M3. A \rightarrow B : \{N_B, N_A\}_{K_B^2}$
 A decifra M2, estrae il nonce N_B e lo reinvia a B insieme al proprio nonce N_A . Questa volta, la cifratura avviene usando la nuova chiave pubblica di B (K_B^2). Reinviando N_B , A dimostra a B di possedere la chiave privata K_A^{-1} (autenticandosi).
4. $M4. B \rightarrow A : \{N_A\}_{K_A^2}$
 B decifra M3, verifica N_B , e reinvia il nonce N_A ad A , cifrandolo con la nuova chiave pubblica di A (K_A^2). Questo dimostra ad A che B ha decifrato correttamente M3 e possiede la nuova chiave privata associata a K_B^2 .

b) NO, E' POSSIBILE MAI IN QUEL MODO. IL PROBLEMA E' CHE MANCA L'IDENTIFICAZIONE DI B NEL PASSO 2

c) $B \rightarrow A : \{B, N_B, K_B^2\}_{K_A^1}$